claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a8557549219ec8cd7.jsonl`
- SHA-256: `497556a20df316f6ca8a1ec76567c96c70145ad4f18a2e5c392600a2ee8b3729`
- Source modified: `2026-06-20T09:01:27+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`

Transcript

1. user (2026-06-20T08:59:56.758Z)

Adversarially verify this review finding against the ACTUAL code in the working tree. Default to 'false-positive' unless you can point to the concrete line(s) and a real failure scenario.
Context: Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master.
The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the worker.
Files changed/added (read them in the WORKING TREE ? changes are uncommitted):
- backend/pipeline/detectors/native_wasb_runner.py (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py (added --device, threaded device into _build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py (_resolve_runner: new 'native' branch + _build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py (_build_native_runner override; WASB substrate only)
- backend/config/models.py (WasbIndexConfig.device + .python_bin; detector_impl doc mentions 'native')
- backend/worker/__main__.py (cmd_train: --trajectory-source {synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)

DESIGN INTENT / CONTRACTS (judge against these):
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner, WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively (no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename '{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) -> identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-platform path handling matters.
- weights/repo/python/scratch come from env first (WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.

FINDING [tests-regress] (major) @ tests/test_native_wasb_runner.py::_drive_success + test_run_predict_builds_native_argv_no_wsl (covering native_wasb_runner.py::run_predict lines 168-171, scratch_dir handling): The scratch_dir -> frames path wiring is completely untested. run_predict computes `scratch = self.cfg.scratch_dir or output_dir` and `frames_dir = os.path.join(scratch, f'{key}_frames')`, then passes it as --frames_out_dir. Tests assert '--

```
frames_out_dir' is PRESENT but never assert its VALUE, and never set scratch_dir, so the env-
driven WASB_SCRATCH/RALLY_SCRATCH/TMPDIR path (a first-class env contract per the design intent)
is never followed end-to-end into the argv. The frames dir that gets --rm -rf'd on cleanup is
likewise never asserted.
WHY: If scratch_dir is dropped from the path join, frames silently land next to the output on
the GPU box (the small system disk) instead of the large scratch volume ? a disk-fill regression
on exactly the long 4K clips this runner targets, and tests stay green. Worse, the cleanup
_rm_rf target could diverge from the extraction target (leaving the GBs of PNGs), undetected.
PROPOSED FIX: Add a test driving run_predict with NativeWasbConfig(scratch_dir='/scratch') (disk
mode) that asserts argv[argv.index('--frames_out_dir')+1] == os.path.join('/scratch',
f'{key}_frames'), and a second with scratch_dir='' asserting the frames dir is under the
abspath'd output_dir. Capture the _rm_rf spy arg and assert it equals that same frames_dir.
```

2. assistant (2026-06-20T08:59:59.194Z)

I'll verify this finding against the actual code. Let me start by reading the relevant files.

3. user (2026-06-20T09:00:00.968Z)

```
On branch feat/native-wasb-runner
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/config/models.py
        modified:   backend/pipeline/detectors/wasb_infer.py
        modified:   backend/pipeline/segmenters/fusion_hybrid.py
        modified:   backend/pipeline/segmenters/trajectory_hybrid.py
        modified:   backend/pipeline/segmenters/wasb_hybrid.py
        modified:   backend/worker/__main__.py
        modified:   tests/test_worker.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        artifacts/
        backend/pipeline/detectors/native_wasb_runner.py
        docs/KHELSUTRA_PRODUCT_OVERVIEW.md
        tests/test_native_wasb_runner.py

no changes added to commit (use "git add" and/or "git commit -a")
---BRANCH---
feat/native-wasb-runner
```

4. user (2026-06-20T09:00:01.545Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
8       transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14          into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15        * weights / repo / python / scratch come from the **environment** first
16          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
```

```
17          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
18
19      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
20      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25      **Acceptance gate (owner / GPU box):** byte-parity ? the SAME clip through the WSL
runner
26      (Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV
and
27      identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
28      env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
29      test suite.
30      """
31      import os
32      import shutil
33      import logging
34      import subprocess
35      from dataclasses import dataclass
36      from typing import Any, Dict, List, Optional, Tuple
37
38      from backend.pipeline.detectors.base import DetectorRunner
39      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
40      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
41
42      logger = logging.getLogger(__name__)
43
44      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
45      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
46      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
47
48
49      @dataclass
50      class NativeWasbConfig:
51          """Resolved settings for a Linux-native WASB run.
52
53          Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
54          over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
55          defaults to ``cuda`` (the WSL byte-parity path).
56          """
57          repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
58          python_bin: str = "python"             # the `wasb` conda env python (invoked by
path, no activate)
59          weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
60          sport: str = "badminton"
61          device: str = "cuda"                   # cuda | mps | cpu
62          scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next
to the output dir
63          timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no
timeout)
64          keep_frames: bool = False              # retain the decoded frame cache after
success (debug only)
65          stream_video: bool = False             # fast path: decode in-process, no PNG
extraction (bit-identical)
66
67          @classmethod
68          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
69              w = (idx_cfg or {}).get("wasb", {}) or {}
70              env = os.environ.get
```

```
71              return cls(
72                  repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
73                  python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
74                  weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
75                  sport=w.get("sport", "badminton"),
76                  device=env("WASB_DEVICE") or w.get("device", "cuda"),
77                  scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
78                  timeout_sec=int(w.get("timeout_sec", 1800)),
79                  keep_frames=bool(w.get("keep_frames", False)),
80                  stream_video=bool(w.get("stream_video", False)),
81              )
82
83
84      class NativeWasbRunner(DetectorRunner):
85          """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
86
87          def __init__(self, cfg: NativeWasbConfig):
88              self.cfg = cfg
89
90          # --- low-level native exec (the single place tests patch)
--------------------------
91          def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
92              logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
93              return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
94                                    timeout=(self.cfg.timeout_sec or None))
95
96          def _repo_src(self) -> str:
97              return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
98
99          def _copy_infer_script(self) -> bool:
100             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
101             repo_src = self._repo_src()
102             try:
103                 os.makedirs(repo_src, exist_ok=True)
104                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
105                 return True
106             except OSError as e:
107                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
108                 return False
109
110         def _rm_rf(self, path: Optional[str]) -> None:
111             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
112             if path:
113                 shutil.rmtree(path, ignore_errors=True)
114
115         def healthcheck(self) -> Tuple[bool, str]:
116             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
117             if not self.cfg.weights_path:
118                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
119             weights = os.path.expanduser(self.cfg.weights_path)
120             if not os.path.exists(weights):
121                 return False, f"WASB weights not found at {weights}"
122             repo_src = self._repo_src()
123             if not os.path.isdir(repo_src):
124                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
125                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
126             code = "import torch; print('torch', torch.__version__, 'cuda',
```

```
torch.cuda.is_available())"
            try:
                res = self._run([self.cfg.python_bin, "-c", code])
            except FileNotFoundError:
                return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
            except subprocess.TimeoutExpired:
                return False, "native torch healthcheck timed out."
            out = (res.stdout or "").strip()
            if res.returncode != 0:
                return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
            if self.cfg.device == "cuda" and "cuda True" not in out:
                return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
            return True, out

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.

            Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
            runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
            same downstream contract, so the parity comparison is apples-to-apples.
            """
            output_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_dir, exist_ok=True)
            # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
            norm = str(video_win_path).replace("\\", "/")
            stem, _ext = os.path.splitext(os.path.basename(norm))
            # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
            key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
            out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

            if not self._copy_infer_script():
                logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
                return None

            weights = os.path.expanduser(self.cfg.weights_path)
            argv = [self.cfg.python_bin, "wasb_infer.py",
                    "--video", os.path.abspath(str(video_win_path))]
            frames_dir: Optional[str] = None
            if self.cfg.stream_video:
                # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
                argv.append("--stream-video")
            else:
                scratch = self.cfg.scratch_dir or output_dir
                frames_dir = os.path.join(scratch, f"{key}_frames")
                argv += ["--frames_out_dir", frames_dir]
            argv += ["--weights", weights,
                     "--sport", self.cfg.sport,
                     "--device", self.cfg.device,
                     "--out", out_csv]

            logger.info("Running native WASB inference (device=%s) on %s ...",
                        self.cfg.device, os.path.basename(norm))
            try:
                res = self._run(argv, cwd=self._repo_src())
            except FileNotFoundError:
                logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
```

```
183                 return None
184             except subprocess.TimeoutExpired:
185                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
186                 return None
187             if res.returncode != 0:
188                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
189                 return None
190             if not os.path.exists(out_csv):
191                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
192                 return None
193             logger.info("WASB trajectory CSV: %s", out_csv)
194
195             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
196             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
197             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
198             if not self.cfg.keep_frames and frames_dir:
199                 logger.info("Cache hygiene: removing frame cache for %s "
200                             "(set indexing.wasb.keep_frames=true to retain).", key)
201                 self._rm_rf(frames_dir)
202             return out_csv
203
204         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
205         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
206         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
207
```

5. user (2026-06-20T09:00:01.814Z)

```
1       """NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).
2
3       The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
4       the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL
runner
5       (no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
6       activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output
CSV
7       name as the WSL runner, threads the device through, and cleans the frame cache on
success.
8       """
9       import os
10      import types
11      from unittest.mock import patch
12
13      import pytest
14
15      from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
16      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin
17
18
19      def _proc(rc=0, stdout="", stderr=""):
20          return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)
21
22
23      # --- identity / no-WSL guarantee
----------------------------------------------------------
24      def test_is_a_detector_runner():
```

```
25          assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)
26
27
28      def test_is_not_a_wsl_runner():
29          """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
30          r = NativeWasbRunner(NativeWasbConfig())
31          assert not isinstance(r, WslCommandMixin)
32          assert not hasattr(r, "_wsl_bash")
33
34
35      def test_reuses_model_agnostic_windowing():
36          from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
37          assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
38          assert NativeWasbRunner.trajectory_to_action_windows is
TrackNetRunner.trajectory_to_action_windows
39
40
41      # --- config resolution (env overrides win over config over default)
---------------------
42      def test_from_indexing_cfg_defaults():
43          cfg = NativeWasbConfig.from_indexing_cfg({})
44          assert cfg.device == "cuda" and cfg.python_bin == "python"
45          assert cfg.weights_path == "" and cfg.repo_dir == "~/models/WASB-SBDT"
46          assert cfg.stream_video is False and cfg.keep_frames is False
47
48
49      def test_from_indexing_cfg_reads_config_block():
50          cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
51              "weights_path": "/m/w.pth.tar", "device": "cpu", "python_bin":
"/envs/wasb/bin/python",
52              "repo_dir": "/m/WASB-SBDT", "stream_video": True, "keep_frames": True, "sport":
"tennis"}})
53          assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
54          assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
55          assert cfg.stream_video is True and cfg.keep_frames is True and cfg.sport ==
"tennis"
56
57
58      def test_env_overrides_beat_config(monkeypatch):
59          monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
60          monkeypatch.setenv("WASB_DEVICE", "mps")
61          monkeypatch.setenv("WASB_PYTHON", "/env/py")
62          monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
63          monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
64          cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
65              "weights_path": "/cfg/w", "device": "cpu", "python_bin": "/cfg/py", "repo_dir":
"/cfg/repo"}})
66          assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
67          assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
68          assert cfg.scratch_dir == "/env/scratch"
69
70
71      # --- healthcheck
------------------------------------------------------------------------
72      def _ok_cfg(tmp_path, **kw):
73          w = tmp_path / "w.pth.tar"
74          w.write_text("weights")
75          (tmp_path / "repo" / "src").mkdir(parents=True)
76          return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)
77
78
79      def test_healthcheck_ok(tmp_path):
80          r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
81          with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
True")):
```

```
            ok, msg = r.healthcheck()
        assert ok and "torch" in msg


    def test_healthcheck_empty_weights():
        ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
        assert not ok and "weights" in msg.lower()


    def test_healthcheck_weights_missing(tmp_path):
        (tmp_path / "repo" / "src").mkdir(parents=True)
        cfg = NativeWasbConfig(weights_path=str(tmp_path / "nope.pth"),
repo_dir=str(tmp_path / "repo"))
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "not found" in msg.lower()


    def test_healthcheck_repo_missing(tmp_path):
        w = tmp_path / "w.pth.tar"
        w.write_text("x")
        cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
        ok, msg = NativeWasbRunner(cfg).healthcheck()
        assert not ok and "repo" in msg.lower()


    def test_healthcheck_torch_import_fails(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path))
        with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError:
torch")):
            ok, msg = r.healthcheck()
        assert not ok and "failed" in msg.lower()


    def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
        with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
            ok, msg = r.healthcheck()
        assert not ok and "cuda" in msg.lower()


    def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
        with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
            ok, msg = r.healthcheck()
        assert ok  # cpu run does not require a GPU


    def test_healthcheck_python_not_found(tmp_path):
        r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
        with patch.object(r, "_run", side_effect=FileNotFoundError()):
            ok, msg = r.healthcheck()
        assert not ok and "python" in msg.lower()


    # --- run_predict
-------------------------------------------------------------------
    def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
        """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
        r = NativeWasbRunner(cfg)
        key = f"clip__{vid[:12]}"
        (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")  # success = CSV
exists
```

```
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
             patch.object(r, "_rm_rf") as rm_spy:
            out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
        argv = run_spy.call_args[0][0]
        cwd = run_spy.call_args.kwargs.get("cwd")
        return r, out, rm_spy, argv, cwd, key


    def test_run_predict_builds_native_argv_no_wsl(tmp_path):
        _, out, rm, argv, cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
        assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
        # No WSL / conda plumbing anywhere in the command.
        assert "wsl" not in argv
        assert not any("conda" in a or "/mnt/" in a for a in argv)
        # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
        assert argv[0] == "python" and argv[1] == "wasb_infer.py"
        assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
        assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
        assert argv[argv.index("--out") + 1] == out
        # Disk mode extracts frames + cleans them on success.
        assert "--frames_out_dir" in argv and "--stream-video" not in argv
        assert cwd.endswith(os.path.join("src"))
        rm.assert_called_once()


    def test_run_predict_stream_mode_no_frames_dir(tmp_path):
        _, out, rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", stream_video=True), tmp_path)
        assert out is not None
        assert "--stream-video" in argv and "--frames_out_dir" not in argv
        rm.assert_not_called()  # streaming never materializes a frame cache


    def test_run_predict_device_threaded_cpu(tmp_path):
        _, _out, _rm, argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path)
        assert argv[argv.index("--device") + 1] == "cpu"


    def test_run_predict_keep_frames_skips_cleanup(tmp_path):
        _, _out, rm, _argv, _cwd, _key = _drive_success(
            NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True), tmp_path)
        rm.assert_not_called()


    def test_run_predict_none_when_copy_infer_fails(tmp_path):
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=False):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_none_when_subprocess_fails(tmp_path):
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(1, stderr="CUDA error")):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_none_when_csv_absent(tmp_path):
        # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
```

```
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", return_value=_proc(0)):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_run_predict_times_out_returns_none(tmp_path):
        import subprocess
        r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
        with patch.object(r, "_copy_infer_script", return_value=True), \
             patch.object(r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x",
timeout=1)):
            assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
is None


    def test_csv_name_matches_wsl_contract(tmp_path):
        """The native CSV name must equal the WSL runner's so downstream + the parity diff
line up."""
        _, out, _rm, _argv, _cwd, key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
        assert os.path.basename(out) == f"{key}_wasb.csv"


    # --- the detector_impl='native' seam (parallels the stub seam)
-------------------------
    def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
        runner, params = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl":
"native"})
        assert isinstance(runner, NativeWasbRunner)
        assert params.get("detector_impl") == "native"


    def test_resolve_runner_native_env_overrides_config(monkeypatch):
        monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
        from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
        runner, _ = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "auto"})
        assert isinstance(runner, NativeWasbRunner)


    def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
        runner, params = FusionSegmenter(None, {})._resolve_runner({"detector_impl":
"native"})
        assert isinstance(runner, NativeWasbRunner)
        assert params.get("fusion_substrate") == "wasb"


    def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
        with pytest.raises(NotImplementedError):
            FusionSegmenter(None, {})._resolve_runner(
                {"detector_impl": "native", "fusion": {"substrate": "tracknet"}})


    def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
        monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
        from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
        with pytest.raises(NotImplementedError):
            TrackNetHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
```

6. assistant (2026-06-20T09:00:05.705Z)

Now let me verify the claim. The finding says the tests assert `--frames_out_dir` is present but never assert its VALUE, and never set scratch_dir. Let me check the test assertions carefully and compare against the actual code path. Let me also check the existing WSL runner for the equivalent contract and see how the WSL runner is tested for the scratch path.

7. user (2026-06-20T09:00:07.125Z)

```
1       """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3       Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9       So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10      with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11      """
12      import os
13      import shlex
14      import logging
15      import subprocess
16      from dataclasses import dataclass
17      from typing import Dict, Any, Optional, Tuple
18
19      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20      # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21      from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24      logger = logging.getLogger(__name__)
25
26      # Windows path to the inference wrapper we stage into WSL.
27      _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30      @dataclass
31      class WasbConfig:
32          """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/WASB-SBDT"
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "wasb"
37          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38          sport: str = "badminton"
39          wsl_stage_dir: str = "~/clips"
40          timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41          keep_frames: bool = False  # retain staged video + frame cache after success (debug
only)
42          min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 =
off)
43          # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44          # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45          # extraction that dominates a long clip. Output is bit-identical to the PNG path
46          # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47          stream_video: bool = False
```

```
48
49          @classmethod
50          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51              w = (idx_cfg or {}).get("wasb", {}) or {}
52              return cls(
53                  distro=w.get("wsl_distro", "Ubuntu"),
54                  repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55                  conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56                  conda_env=w.get("conda_env", "wasb"),
57                  weights_path=w.get("weights_path",
58                                     "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59                  sport=w.get("sport", "badminton"),
60                  wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61                  timeout_sec=int(w.get("timeout_sec", 1800)),
62                  keep_frames=bool(w.get("keep_frames", False)),
63                  min_free_gb=float(w.get("min_free_gb", 0.0)),
64                  stream_video=bool(w.get("stream_video", False)),
65              )
66
67
68      class WasbRunner(WslCommandMixin, DetectorRunner):
69          def __init__(self, cfg: WasbConfig):
70              self.cfg = cfg
71
72          def healthcheck(self) -> Tuple[bool, str]:
73              """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74              cmd = (f"conda activate {self.cfg.conda_env} && "
75                     f"python -c \"import torch; print('torch', torch.__version__, "
76                     f"'cuda', torch.cuda.is_available())\"")
77              try:
78                  res = self._wsl_bash(cmd)
79              except FileNotFoundError:
80                  return False, "`wsl` not found ? Windows + WSL2 required."
81              except subprocess.TimeoutExpired:
82                  return False, "WSL healthcheck timed out."
83              out = (res.stdout or "").strip()
84              if res.returncode != 0:
85                  return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86              return True, out
87
88          def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89              src_mnt = to_wsl_mnt_path(video_win_path)
90              reason = self.wsl_source_unreadable_reason(src_mnt)
91              if reason:
92                  logger.error("Cannot stage video into WSL: %s", reason)
93                  return None
94              dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95              cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96              try:
97                  res = self._wsl_bash(cmd)
98              except subprocess.TimeoutExpired:
99                  logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
```

```
OK"
            res = self._wsl_bash(cmd)
            return res.returncode == 0 and "OK" in (res.stdout or "")

        def run_predict(self, video_win_path: str, output_win_dir: str,
                        video_id: Optional[str] = None) -> Optional[str]:
            """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.

            All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
            two videos that share a filename (e.g. two ``match.mp4`` from different folders)
            can never reuse each other's staged frames, resumable cache, or CSV.
            """
            # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
            # translation: to_wsl_mnt_path passes relative paths through unchanged, so
            # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
            # the CSV landed inside WSL while Windows polled the repo-relative path.
            output_win_dir = os.path.abspath(output_win_dir)
            os.makedirs(output_win_dir, exist_ok=True)
            # Normalize for cross-platform basename (tests use Windows paths on Linux).
            video_win_path = str(video_win_path).replace("\\", "/")
            base = os.path.basename(video_win_path)
            stem, ext = os.path.splitext(base)
            # Unique, content-derived key: same filename + different bytes -> different key.
            key = f"{stem}__{video_id[:12]}" if video_id else stem
            wsl_out_dir = to_wsl_mnt_path(output_win_dir)
            expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")

            if not self._stage_infer_script():
                logger.error("Could not stage wasb_infer.py into WSL.")
                return None
            staged_video = self._stage_video(video_win_path, f"{key}{ext}")
            if staged_video is None:
                return None
            frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"

            # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
            # WSL stage filesystem is already low so the user can clean up before it fills.
            # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
            if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
                free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
                if free is not None and free < self.cfg.min_free_gb:
                    logger.warning(
                        "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
                        "may fill the disk ? consider running tools/cleanup_caches.py.",
                        free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

            # Fast path: stream-decode the video in-process (no PNG extraction). Output is
            # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
            if self.cfg.stream_video:
                frame_args = "--stream-video "
            else:
                frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
            cmd = (
                f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
                f"python wasb_infer.py "
                f"--video {wsl_tilde_quote(staged_video)} "
                f"{frame_args}"
                f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
                f"--sport {shlex.quote(self.cfg.sport)} "
                f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
            )
            logger.info("Running WASB inference on %s ...", base)
```

```
171             try:
172                 res = self._wsl_bash(cmd)
173             except subprocess.TimeoutExpired:
174                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175                 return None
176             if res.returncode != 0:
177                 logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178                 return None
179             if not os.path.exists(expected_csv_win):
180                 logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181                 return None
182             logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184             # Success ? the trajectory CSV is the durable output; the staged video copy and
185             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187             # so the frame cache + detector cache remain for a resumable re-run.
188             # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189             if not self.cfg.keep_frames:
190                 stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191                 logger.info("Cache hygiene: removing staged video%s for %s "
192                             "(set indexing.wasb.keep_frames=true to retain).",
193                             "" if self.cfg.stream_video else " + frame cache", key)
194                 self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200
```

8. user (2026-06-20T09:00:07.917Z)

```
test_calibrate_wasb.py
test_config_wasb.py
test_export_wasb_segments.py
test_native_wasb_runner.py
test_wasb_cache.py
test_wasb_runner.py
```

9. assistant (2026-06-20T09:00:11.825Z)

Now let me verify the precise claims in the finding by examining the test assertions closely. The finding makes specific claims:

1. Tests assert `--frames_out_dir` is PRESENT but never assert its VALUE.
2. Tests never set `scratch_dir`, so the env-driven scratch path is never followed end-to-end into argv.
3. The frames dir that gets `_rm_rf`'d on cleanup is never asserted (the `rm_spy` arg is never checked).

Let me grep the test file for these assertions to confirm.

10. user (2026-06-20T09:00:18.201Z)

```
63:    monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
68:    assert cfg.scratch_dir == "/env/scratch"
136:    """Drive run_predict to success with the subprocess mocked; returns (runner, out,
```

```
rm_spy, argv, cwd)."""
142:         patch.object(r, "_rm_rf") as rm_spy:
144:    argv = run_spy.call_args[0][0]
145:    cwd = run_spy.call_args.kwargs.get("cwd")
146:    return r, out, rm_spy, argv, cwd, key
161:    assert "--frames_out_dir" in argv and "--stream-video" not in argv
163:    rm.assert_called_once()
170:    assert "--stream-video" in argv and "--frames_out_dir" not in argv
171:    rm.assert_not_called()  # streaming never materializes a frame cache
183:    rm.assert_not_called()
```

11. assistant (2026-06-20T09:00:26.791Z)

The grep confirms the finding's claims about the test gaps. Let me verify each precise claim:

Claim 1: Tests assert `--frames_out_dir` is PRESENT but never assert its VALUE.
- Line 161: `assert "--frames_out_dir" in argv and "--stream-video" not in argv` ? presence only.
- Line 170: `assert "--stream-video" in argv and "--frames_out_dir" not in argv` ? absence (stream mode).
- No test does `argv[argv.index("--frames_out_dir") + 1] == <value>`. Confirmed.

Claim 2: Tests never set `scratch_dir` in a run_predict drive.
- Line 68: `assert cfg.scratch_dir == "/env/scratch"` ? this only tests config *resolution* (`from_indexing_cfg`), NOT that the resolved scratch_dir flows into the argv `--frames_out_dir` value via run_predict.
- The `_drive_success` helper (lines 135-146) never passes `scratch_dir`. All run_predict tests use `NativeWasbConfig(weights_path=..., ...)` with `scratch_dir` defaulting to `""`. Confirmed.

Claim 3: The `_rm_rf` target value is never asserted.
- Line 163: `rm.assert_called_once()` ? call count only.
- Line 171/183: `rm.assert_not_called()`.
- No test checks `rm.call_args` to confirm the deleted path equals the frames_dir passed as `--frames_out_dir`. Confirmed.

Now let me verify the WHY ? the failure scenario. Let me check the actual code path at lines 169-171 and 198-201 to confirm the scratch_dir ? frames_dir ? argv ? cleanup wiring is real and that a regression would be undetected.

12. user (2026-06-20T09:00:27.581Z)

```
164             frames_dir: Optional[str] = None
165             if self.cfg.stream_video:
166                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
167                 argv.append("--stream-video")
168             else:
169                 scratch = self.cfg.scratch_dir or output_dir
170                 frames_dir = os.path.join(scratch, f"{key}_frames")
171                 argv += ["--frames_out_dir", frames_dir]
172             argv += ["--weights", weights,
173                      "--sport", self.cfg.sport,
174                      "--device", self.cfg.device,
175                      "--out", out_csv]
176
177             logger.info("Running native WASB inference (device=%s) on %s ...",
178                         self.cfg.device, os.path.basename(norm))
179             try:
180                 res = self._run(argv, cwd=self._repo_src())
181             except FileNotFoundError:
182                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
183                 return None
184             except subprocess.TimeoutExpired:
185                 logger.error("WASB native inference timed out after %ss.",
```

```
self.cfg.timeout_sec)
186                 return None
187             if res.returncode != 0:
188                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
189                 return None
190             if not os.path.exists(out_csv):
191                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
192                 return None
193             logger.info("WASB trajectory CSV: %s", out_csv)
194
195             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
196             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
197             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
198             if not self.cfg.keep_frames and frames_dir:
199                 logger.info("Cache hygiene: removing frame cache for %s "
200                             "(set indexing.wasb.keep_frames=true to retain).", key)
201                 self._rm_rf(frames_dir)
202             return out_csv
```

13. assistant (2026-06-20T09:00:32.198Z)

The code is exactly as the finding describes. Now let me assess the **severity and the "real failure scenario"** rigorously, since the finding is a *test-quality* (coverage gap) finding, not a code bug. Let me confirm the regression it describes would truly stay green by mentally tracing a hypothetical regression, and verify there's no OTHER test that would catch it. Let me run the actual test file to confirm current state, and check whether any test indirectly covers the scratch path value.

14. user (2026-06-20T09:00:41.161Z)

```
............................                                             [100%]
28 passed in 6.01s
```

15. assistant (2026-06-20T09:00:48.194Z)

All 28 tests pass. Now let me confirm the regression scenarios the finding describes would actually slip through. Let me trace both hypothetical regressions:

Regression A: `scratch = self.cfg.scratch_dir or output_dir` ? drops scratch, becomes just `output_dir`. In that case, with the default config (`scratch_dir=""`), `scratch` would already be `output_dir` anyway ? so default-config tests are blind by construction. With a configured `scratch_dir='/scratch'`, the frames_dir would wrongly land under output_dir. But **no run_predict test sets scratch_dir**, so this regression is undetected. Confirmed.

Regression B: cleanup target diverges from extraction target. If `_rm_rf(frames_dir)` were changed to delete a different/wrong path, line 163's `rm.assert_called_once()` would still pass (it only checks call count, not the argument). Confirmed.

Let me double-check there's no value assertion I missed by reading the full assertion block of the relevant test once more, and verify the WSL runner's test suite for comparison (to confirm this is a genuine coverage delta, not the repo's accepted norm).

16. user (2026-06-20T09:00:50.159Z)

```
57:# --- cache hygiene (keep_frames / min_free_gb / shared WSL helpers) ----------
60:    cfg = WasbConfig.from_indexing_cfg({"wasb": {"keep_frames": True, "min_free_gb": 25}})
61:    assert cfg.keep_frames is True and cfg.min_free_gb == 25.0
63:    assert default.keep_frames is False and default.min_free_gb == 0.0
66:def test_wsl_rm_rf_builds_rm_command():
```

```
69:        r._wsl_rm_rf(["~/clips/a_frames", "~/clips/a.mp4"])
70:    inner = wb.call_args[0][0]
71:    assert inner.startswith("rm -rf ") and "a_frames" in inner and "a.mp4" in inner
74:def test_wsl_rm_rf_noop_on_empty():
77:        r._wsl_rm_rf([])
97:         patch.object(r, "_wsl_rm_rf") as rm:
103:def test_run_predict_cleans_frames_and_staged_video_on_success(tmp_path):
105:    rm.assert_called_once()
106:    purged = rm.call_args[0][0]
107:    assert f"~/clips/{key}_frames" in purged and f"~/clips/{key}.mp4" in purged
110:def test_run_predict_keeps_frames_when_keep_frames_true(tmp_path):
111:    rm, _ = _run_predict_to_success(WasbConfig(keep_frames=True), tmp_path)
```

17. assistant (2026-06-20T09:00:56.080Z)

This is the key comparison. The **WSL runner's test suite** (`test_wasb_runner.py`) ? the established sibling this native runner is modeled on ? DOES assert the cleanup target value:

- Line 105: `rm.assert_called_once()`
- Lines 106-107: `purged = rm.call_args[0][0]` then `assert f"~/clips/{key}_frames" in purged and f"~/clips/{key}.mp4" in purged`

So the repo's own convention/precedent for the sibling runner **does** assert the `_rm_rf` argument value, while the new native runner test only asserts `rm.assert_called_once()` (line 163) and never inspects `rm.call_args`. This confirms the finding is identifying a genuine coverage regression relative to the established sibling pattern.

Let me also confirm the WSL test asserts the frames path in the command argv (the equivalent of the `--frames_out_dir` value check) to gauge whether the value-assertion gap is also a delta.

18. user (2026-06-20T09:00:57.065Z)

```
80
81      def test_wsl_free_gb_parses_bytes_else_none():
82          r = WasbRunner(WasbConfig())
83          with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout=str(50 * 1024 ** 3) +
"\n")):
84              assert abs(r._wsl_free_gb("~/clips") - 50.0) < 0.01
85          with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="garbage")):
86              assert r._wsl_free_gb("~/clips") is None
87
88
89      def _run_predict_to_success(cfg, tmp_path, vid="abc123abc123"):
90          """Drive run_predict to its success branch with WSL fully mocked. Returns (rm_spy,
key)."""
91          r = WasbRunner(cfg)
92          key = f"clip__{vid[:12]}"
93          (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")  # success = CSV
exists
94          with patch.object(r, "_stage_infer_script", return_value=True), \
95               patch.object(r, "_stage_video", return_value=f"~/clips/{key}.mp4"), \
96               patch.object(r, "_wsl_bash", return_value=_proc(0)), \
97               patch.object(r, "_wsl_rm_rf") as rm:
98              out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
99          assert out is not None
100         return rm, key
101
102
103     def test_run_predict_cleans_frames_and_staged_video_on_success(tmp_path):
104         rm, key = _run_predict_to_success(WasbConfig(), tmp_path)
105         rm.assert_called_once()
106         purged = rm.call_args[0][0]
107         assert f"~/clips/{key}_frames" in purged and f"~/clips/{key}.mp4" in purged
108
109
```

```
110     def test_run_predict_keeps_frames_when_keep_frames_true(tmp_path):
111         rm, _ = _run_predict_to_success(WasbConfig(keep_frames=True), tmp_path)
112         rm.assert_not_called()
113
```

19. assistant (2026-06-20T09:01:12.194Z)

I now have everything I need. My verification is complete.

Summary of verification

The finding's factual claims are all confirmed against the working-tree code:

1. **`--frames_out_dir` value is never asserted.** `tests/test_native_wasb_runner.py:161` asserts only `"--frames_out_dir" in argv`. No test does `argv[argv.index("--frames_out_dir")+1] == <expected>`. Confirmed.

2. **`scratch_dir` is never exercised through `run_predict`.** The `_drive_success` helper (lines 135-146) never passes `scratch_dir`; every run_predict test leaves it at default `""`. The only scratch coverage (line 68) tests `from_indexing_cfg` resolution, not the `scratch ? frames_dir ? argv` path in `run_predict` (native_wasb_runner.py:169-171). With the default `scratch_dir=""`, `scratch = self.cfg.scratch_dir or output_dir` already collapses to `output_dir`, so a regression that drops `scratch_dir` from the join is invisible to the existing tests. Confirmed.

3. **The `_rm_rf` cleanup target value is never asserted.** Line 163 is `rm.assert_called_once()` (count only); `rm.call_args` is never inspected. A cleanup target that diverged from the extraction `frames_dir` would leave the test green. Confirmed.

Failure scenario is real and concrete: native_wasb_runner.py targets long 4K clips on a Linux GPU box; `scratch_dir` is a documented first-class env contract (`WASB_SCRATCH`/`RALLY_SCRATCH`/`TMPDIR`, module docstring lines 15-17, config line 77) whose entire purpose is to keep the GBs of extracted PNG frames off the small system disk. A regression dropping `scratch` from the path join, or pointing `_rm_rf` at the wrong dir, is a disk-fill / leaked-frames regression that the current tests cannot catch.

Repo precedent corroborates this is a genuine coverage gap, not the accepted norm: the sibling WSL runner ? which this native runner is explicitly modeled on ? DOES assert the cleanup target value (`test_wasb_runner.py:106-107`: `purged = rm.call_args[0][0]; assert f"~/clips/{key}_frames" in purged ...`). The new native test regressed away that assertion.

Severity calibration: This is a test-coverage gap, not a live code bug ? the production code at lines 169-171 and 198-201 is currently correct, and the byte-parity acceptance gate is explicitly owner-verified on the GPU box (out of scope for this suite). The undetected-regression risk is real and matches a documented disk-fill failure mode the repo cares about, but it is latent (requires a future edit to manifest) and the proposed fix is a small additive test. That fits "minor" better than "major" ? the reviewer's "major" overstates it given the code is correct today and the fix is purely additive test hardening.

20. user (2026-06-20T09:01:21.710Z)

Structured output provided successfully

21. assistant (2026-06-20T09:01:27.843Z)

Verification complete. The finding is **real** but I've corrected the severity from major to **minor**.

All three of the finding's factual claims hold against the working tree:
- `tests/test_native_wasb_runner.py:161` asserts only the presence of `--frames_out_dir`, never its value.
- No `run_predict` test sets `scratch_dir`; the only scratch coverage (line 68) tests config resolution, not the `scratch ? frames_dir ? argv` wiring at `native_wasb_runner.py:169-171`. With the default `scratch_dir=""`, `scratch or output_dir` already collapses to `output_dir`, so

a regression dropping `scratch_dir` from the join stays green.
- Cleanup is asserted by count only (line 163 `rm.assert_called_once()`); `rm.call_args` is never inspected.

The repo's own precedent corroborates this is a genuine gap: the sibling WSL runner's test (`tests/test_wasb_runner.py:106-107`) DOES assert the `_rm_rf` target value, which the new native test regressed away.

I downgraded to minor because the production code is correct today, the byte-parity gate is owner-verified on the GPU box (out of scope for this suite), and the risk is latent ? it requires a future edit to manifest, and the fix is a small additive test.